

Analysis: Load Testing

Understanding the sample cases

In order to solve this problem, one first needs to get a feeling of what's asked, and looking at the sample cases is a good way to achieve that.

Take a look at the first sample case. The answer given is 2. How do we achieve the result with just 2 loadtests? Let's make a couple guesses. Suppose we do a loadtest that checks if the site supports 100 users. If we learn that the site can't support 100 users, then we're done: we know we can support 50 users but can't support 100 which is 50×2 . However, if we learn that the site can actually support 100 users, we have a very difficult task at hand now: we have only one loadtest left, and we know that the site can support 100 users but can't support 700. Is it possible to solve it?

Suppose we now loadtest to check if the site supports 300 users. If we learn that the site can't support 300 users, then we've failed to solve the problem: we know we can support 100, but can't support 300 - but 300 is more than 100×2 , so we don't have enough knowledge. Moreover, this actually helps us to prove that our loadtest must test for 200 users or less, otherwise we will hit the same issue.

Now we know that our second loadtest must use at most 200 users. But even when it's exactly 200, suppose we learn that our site can actually support all of them. Then we've failed to solve the problem again: we know we can support 200, but can't support 700 - but 700 is more than 200×2 .

So there's no good choice for our second loadtest. It means that the choice of the first loadtest was wrong - 100 users is too small for it.

What have we learned?

However, we've learned an important lesson in our failed attempt to understand the example case: when we have only one loadtest left, and we know that the site can support L people but can't support P people, we must loadtest with such number X that $L \times C \geq X$, and at the same time $X \times C \geq P$. The first inequality will help us solve the problem when the loadtest fails, and the second one is helpful if the loadtest succeeds.

Since there's no such number X for $L=100$, $P=700$, $C=2$, our first attempt above has failed.

The question now is: how to check if such X exists? From the first equation, we get $X \leq L \times C$. From the second one, we get $X \geq P/C$. Such X exists if and only if $L \times C \geq P/C$, which means $L \times C^2 \geq P$. Forgetting the formulas, the upper bound of our range should be at most C^2 times more than the lower bound. In that case, we can just take $X=L \times C$ for our only loadtest.

The second attempt at understanding the first sample case

Equipped with this knowledge, we get back to the first sample case. 100 was wrong since $100 \times 2^2 = 100 \times 4 < 700$. Maybe we should loadtest for 300 people first? If the loadtest succeeds, then we will have one loadtest left, 300 people OK, 700 people not OK, and since $300 \times 4 \geq 700$, we can solve the problem. However, what if the loadtest doesn't succeed? We know that our system can support 50 people but can't support 300 people and have only one loadtest left. Since $50 \times 4 < 300$, we can't do that. So the choice of 300 was also wrong.

What if we try 200 as the first loadtest? In case it succeeds, we get one loadtest, 200 OK, 700 not OK, $200 \cdot 4 \geq 700$ - we can do that. In case it fails, we get 50 OK, 200 not OK, $50 \cdot 4 \geq 200$ - we can do that as well. So we've finally figured out the algorithm to solve the first sample case using just 2 loadtests:

```
Loadtest for 200 people. If the site can support 200 people:
    Loadtest for 400 people.
If the site can't support 200 people:
    Loadtest for 100 people.
```

What have we learned?

So how do we figure out if two loadtests are enough? This is actually surprisingly similar to the study of the one loadtest case.

When we have two loadtests left, and we know that the site can support L people but can't support P people, we must loadtest with such number X that $L \cdot C^2 \geq X$, and at the same time $X \cdot C^2 \geq P$. The first inequality will help us solve the problem using one remaining loadtest when the loadtest fails, and the second one is helpful if the loadtest succeeds.

Using the same argument as above, one can see that such number X exists if and only if $L \cdot C^4 \geq P$ (we got C^4 as $C^2 \cdot C^2$).

More loadtests?

Now it's not so hard to figure out what happens with more than two loadtests. It's possible to solve the problem using three loadtests if and only if $L \cdot C^8 \geq P$. For four loadtests, we get $L \cdot C^{16} \geq P$. And so on. That pretty much describes the solution for this problem.

Understanding the sample cases, attempt 3

Now we can finally figure out the algorithm to solve the third sample case: $L=1$, $P=1000$, $C=2$. In order to do this in four loadtests, our first loadtest can be for $L \cdot C^8 = 256$:

```
Loadtest for 256 people. If the site can support them:
    Loadtest for 512 people.
If we can't support 256 people:
    Loadtest for 16 people. If the site can support them:
        Loadtest for 64 people. If the site can support them:
            Loadtest for 128 people.
        If we can't support 64 people:
            Loadtest for 32 people.
    If we can't support 16 people:
        Loadtest for 4 people. If the site can support them:
            Loadtest for 8 people.
        If we can't support 4 people:
            Loadtest for 2 people.
```

This looks quite similar to the binary search algorithm, but performed on exponential scale.

Conclusion

We started solving this problem by trying to understand the answers for the sample cases, and by the time we actually understood them, we already have a complete solution. The only

remaining thing is to implement the solution carefully avoiding the integer overflow issues.

Analysis: Making Chess Boards

Finding the largest chess board

First, we need a quick way to find the largest chess board. There is classic dynamic programming trick that goes like this. Let's compute, for each cell (i, j) , the size of the largest square whose bottom-right corner is (i, j) . Let's call this value $\text{larg}[i][j]$. It is easy to compute $\text{larg}[i][0]$ and $\text{larg}[0][j]$ -- they are always 1. For any other cell, the value of $\text{larg}[i][j]$ is always at least 1, and it is larger only if the following condition holds:

```
if (board[i - 1][j] != board[i][j] &&
    board[i][j - 1] != board[i][j] &&
    board[i - 1][j - 1] == board[i][j]) {
    larg[i][j] = 1 + min(larg[i - 1][j],
                        larg[i][j - 1],
                        larg[i - 1][j - 1]);
}
```

In a single, linear-time, row-by-row scan, we can compute the values of $\text{larg}[][]$ for all cells.

Finding the chess board to remove

Now that we have $\text{larg}[][]$, it is very easy to find the first chess board that we should cut out. Its bottom-right corner is in the cell that has the largest possible value in $\text{larg}[][]$. If there are several such cells, we use the tie-breaking rules described in the problem and choose the one that comes first in lexicographic order of (i, j) .

We can do this in linear time by scanning $\text{larg}[][]$, but since we will have to do this many times, it is better to make a heap of triples of the form

```
(-larg[i][j], i, j)
```

and take the smallest element from that heap. This way, we are sorting all cells by decreasing size, then by increasing row, then by increasing column. As long as we can update this heap efficiently after cutting out a chess board, we can always retrieve the smallest element in $O(\log(m*n))$ time. We could also use a balanced binary search tree instead of a heap.

Removing the chess board and updating $\text{larg}[][]$

Consider removing the first 6x6 chess board from the example input described in the problem statement. How should we update $\text{larg}[][]$? First of all, we can fill the 6x6 square of cells with zeros because there are no more chess boards to be removed from those locations. But that is not all. There are other cells that might need to be updated. Where are they, and how many of them are there?

Naively, we can simply recompute the values of all non-zero cells in $\text{larg}[][]$ and continue. If we do that, we will have a $O(m^2*n^2)$ algorithm, which is too slow.

First of all, notice that we do not need to update rows above or to the left of the 6x6 square. Any square chess boards whose bottom-right corners are in those areas still exist and can be cut out later. The only chess boards that we need to worry about are those that overlap the 6x6 board that we have just removed. Also, notice that we have just removed the largest possible chess

board, so we only need to care about remaining boards of size 6 or smaller. Where can their bottom-right corners lie in order for those boards to overlap *our* board? They must be in the 12x12 square whose center is at (i, j) -- the bottom-right corner of our board.

That's an area of size $4*6^2$. In fact, whenever we remove a board of size k-by-k, we only need to update an area of `larg[][]` of size at most 2k-by-2k. Since we can only remove each cell at most once, all of the updating work requires linear time in total; $4*m*n$ updates, to be precise.

Updating the heap

Each time we update `larg[][]`, we must also update the heap that lets us find the next board to remove. This means finding and removing an old entry, as well as inserting a new entry. With pointers from cells to heap elements, or by using a balanced binary search tree, both steps can be done in $O(m*n)$ time.

In total, this algorithm runs in $O(n*m*\log(n*m))$ time, which is plenty fast for the problem's constraints.

Analysis: Rope Intranet

This problem was easier than it could have been, since the constraints don't require you to write an efficient solution. To solve it you can simply iterate through each pair of ropes and test if they intersect. Checking for intersection can be done in various ways. One way is to write the two line equations and then solve a system of linear equations with two variables to find the intersection point. An easier solution is to simply check if the order of the ends of the pair of ropes on the first building is the opposite of the order of the ends of the ropes on the second building. This translates to the code:

```
(A[i] - A[j]) * (B[i] - B[j]) < 0
```

This algorithm takes $O(n^2)$ and it's fast enough to solve our problem.

This problem is very similar to the classic one which asks for the number of inversions within a given permutation. An inversion for a permutation p is a pair of two indexes $i < j$ such that $p_i > p_j$. Let's see why these problems are equivalent. If ra is the rank of $A[i]$ when we sort A and rb is the rank of $B[i]$ when we sort B , then the ropes problem becomes the inversion count problem on the permutation p where $p_{ra} = rb$ for each i .

This new problem is a good application for divide and conquer algorithms, and can be solved in $O(n \log n)$ time. Merge sort can be adapted nicely to not only sort an array, but count the number of inversions as well. Other solutions use data structures like segment trees, augmented balanced binary search trees or augmented skip lists.

[Number of inversions in a permutation](#)